Bioinformatics, 2026, 1–4
Advance Access Publication Date: 2026
Applications Note

Sequence analysis

adaptiSeq: complete, verified download of public sequencing accession lists

Rasman Mubtasim Swargo[1], **Nieb Hasan Neom**[1] **and Md Arifuzzaman**[1,*]

[1]Department of Computer Science, Missouri University of Science and Technology, Rolla, MO, USA
[*]To whom correspondence should be addressed. E-mail: marifuzzaman@mst.edu
Associate Editor: XXXXXXX

Received on XXXXX; revised on XXXXX; accepted on XXXXX

Abstract

Summary: Reanalysis of public sequencing data starts from a list of accessions, often hundreds of runs. Existing downloaders fetch such a list one run at a time over one connection, and some report success while runs are missing or were silently replaced by files that cannot be checked against the archive. adaptiSeq is a Python library and command-line tool, a reimplementation of iSeq, that resolves accessions from GSA, SRA, ENA, DDBJ and GEO in parallel under per-archive request limits, downloads files as resumable byte ranges over many connections, checks every file against the archive's MD5 as it lands, and exits non-zero when a run is not delivered. On a 241-run ENA project, the fastest other tool that delivered every run was 6.1 times slower than adaptiSeq with default settings in the median repetition; adaptiSeq never exited 0 with a run missing, against 8 of 40 benchmark rows for iSeq, Kingfisher and fastq-dl. Resolution, transfer and verification are also available as Python functions, and several controllers for the number of concurrent files are built in.

Availability and implementation: adaptiSeq is written in Python (≥3.10) and released under the MIT licence at `https://github.com/swargo98/adaptiSeq` and `https://pypi.org/project/adaptiseq/`. Benchmark scripts, accession lists and result tables are in the repository.

Contact: marifuzzaman@mst.edu

Supplementary information: Supplementary data are available at *Bioinformatics* online.

1 Introduction

The Sequence Read Archive (SRA; Katz *et al.*, 2022), the European Nucleotide Archive (ENA; Yuan *et al.*, 2026), the DDBJ Sequence Read Archive (Ara *et al.*, 2026) and the Genome Sequence Archive (GSA; Chen *et al.*, 2021) hold tens of millions of sequencing runs, and reanalysis of deposited runs is routine. The unit of work is rarely one file: a pipeline is given a project or a list of run accessions and must obtain every run, intact, before analysis can start.

Several tools download by accession. iSeq (Chao *et al.*, 2024) covers five databases with one command-line interface and verifies checksums. Kingfisher (Woodcroft *et al.*, 2024) tries several sources for each run, fastq-dl (Petit *et al.*, 2026) retrieves FASTQ files from ENA or SRA, nf-core/fetchngs (Patel *et al.*, 2026; Ewels *et al.*, 2020) wraps retrieval in a Nextflow workflow, and pysradb (Choudhary, 2019) exposes metadata and downloads to Python. Apart from fetchngs and pysradb with a thread count, these tools handle the runs of a list sequentially. That matters because ENA serves each TCP connection at only a few MB/s, so one connection leaves most of the path idle. It also matters because archive services fail in ways that tools can mistake for an answer. In a 17-day probe of the ENA portal API from a compute node, 3.1% of 3,420 queries failed outright, and a tool that reads a failed query as "run not in ENA" falls back to another source whose files cannot be checked against ENA's checksum.

We present adaptiSeq, a Python reimplementation and extension of iSeq built for accession lists. It keeps iSeq's database coverage, options and integrity checks, and changes how the bytes arrive: concurrent resolution, a segmented and resumable transfer engine ported from FastBioDL (Swargo *et al.*, 2025), in-pool checksum verification, and an exit status that reflects what was delivered.

2 Implementation

Resolution. Run metadata and file locations are obtained in iSeq's order: the ENA Portal API, NCBI for runs ENA does not mirror, and the GSA interfaces for CRA/CRR accessions. Lookups for a list run in a thread pool (`-meta-jobs`), each endpoint behind its own request limiter (8 requests/s for ENA, 5 for GSA, 3 for NCBI, 10 with an API key). A failed ENA query is retried with backoff and is not read as "not in ENA" until about three minutes of continuous failure; any fallback to SRA is announced, recorded as unverified, and can be refused (`-no-sra-fallback`).

Transfer. The resolved files form one queue served by up to `-j` workers (default 20). Each file is split into byte ranges (one per 512 MiB, at most eight), fetched over HTTPS where the host honours ranges and otherwise over FTP, and written at its offset into a `.part` file with progress in a sidecar, so an interrupted transfer resumes per range. Idle connections take over the remainder of the largest outstanding range, so connections stay busy until the last byte. A per-host guard reads a refused connection while others are open as a ceiling to wait at, and halves its limit only on HTTP 429/503.

Table 1. Feature comparison, from each tool's help output, source and documentation at the versions shown.

	adapti-Seq 0.1.12	iSeq 1.9.9	King-fisher 0.5.0	fastq-dl 4.0.1	fetch-ngs 1.13.0	pysra-db 2.5.1
Accession list in one call	✓	✓	✓	–	✓	project
Several runs at once	✓	–	–	–	✓	✓
Rate-limited parallel resolution	✓	–	–	–	no cap	–
MD5 check	✓	✓	optional	✓	✓	–
Resume partial file	✓	✓	–	–	task	✓
Merge runs	✓	✓	–	✓	–	–
Python API	✓	–	✓	–	–	✓
GSA / GEO	✓/✓	✓/✓	exp./–	–/–	–/✓	–/✓

Integrity. Each finished file is hashed inside the worker pool and compared with the MD5 the archive publishes; a mismatching file is deleted and re-queued, so a corrupt copy is fetched again in parallel with the rest of the list. A run that cannot be verified after three rounds is written to `fail.log` and the exit status is non-zero. Runs with three FASTQ files are downloaded and verified in full.

Interfaces. The command line follows iSeq's (`adaptiseq -i accessions.txt -g`). The same operations are importable, return values and raise typed exceptions, and work inside a running event loop:

```
from adaptiseq import fetch, get_metadata
rows = get_metadata("PRJNA916347")
res = fetch("accessions.txt", outdir="d",
gzip=True)
if res.failed:  print(res.fail_ids)
```

adaptiSeq also includes five controllers for the number of files in flight (`-controller`); the default searches downward from `-j` and commits to the count with the highest measured throughput, and the others (e.g. one that uses the fewest connections) are available to users with other goals. Table 1 compares adaptiSeq with the other tools.

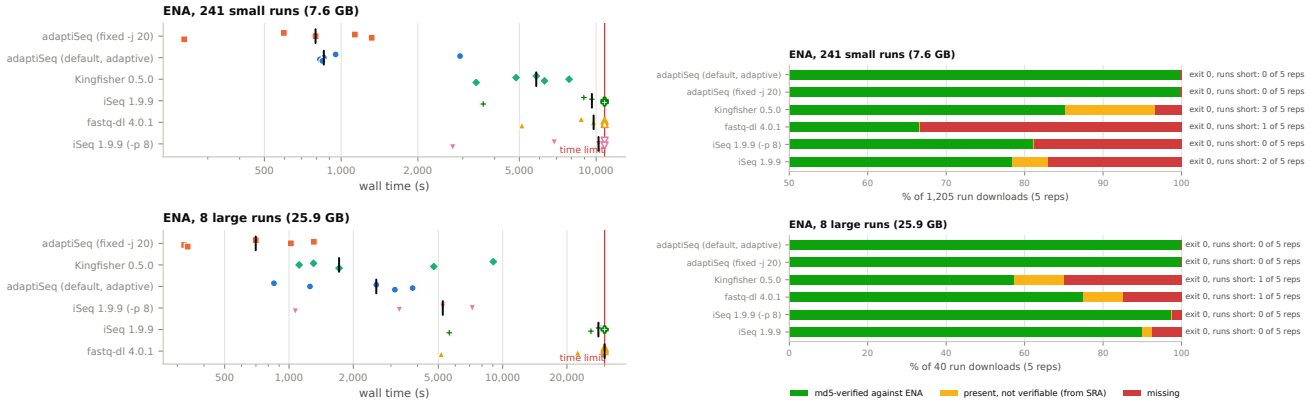


Figure 1. Five repetitions of two ENA lists on SDSC Expanse. **(a)** Wall time of every arm, small runs (top, 241 runs) and large runs (bottom, 8 runs); filled markers finished, hollow markers reached the time limit (lower bounds), bars are medians, the horizontal axis is logarithmic. **(b)** Run downloads over five repetitions split into MD5-verified against ENA, present but not verifiable (fetched from SRA and converted), and missing; the right-hand notes count repetitions in which the tool exited 0 although runs were short.

3 Benchmark

We compared adaptiSeq 0.1.12 with iSeq 1.9.9 (default, and `-p 8`), Kingfisher 0.5.0 and fastq-dl 4.0.1 on two public ENA lists chosen to stress different costs: BioProject PRJNA916347 (241 runs, 321 files, 7.6 GB, 40 runs with three FASTQ files; per-run overhead dominates) and eight runs of PRJNA762469 (16 files, 25.9 GB; bandwidth dominates). Every tool was asked for gzip-compressed FASTQ with checksum verification on. adaptiSeq ran with defaults (`adaptiseq -i LIST -g`) and with a fixed pool (`-no-adaptive -j 20`). Each arm ran five times on an exclusive node of SDSC Expanse (Strande *et al.*, 2021), one arm at a time, in an order drawn afresh for each repetition, with a time limit of 3 h (small runs) or 8.3 h (large runs). Afterwards an independent verifier hashed the output and matched it to a manifest taken from ENA; a run counts as complete only if every file matches, whatever the tool's exit status. Throughput is verified bytes over elapsed time, including resolution and verification. We report medians over repetitions and, for speed, the median of the paired ratio within a repetition.

Speed. On the 241-run list, adaptiSeq's median wall time was 855 s with defaults (8.9 MB/s) and 794 s with the fixed pool (9.5 MB/s) (Fig. 1a). The other tools' medians never exceeded 1.3 MB/s. The fastest other tool that delivered every run was, per repetition, a median of 6.1 times slower than the default and 9.8 times slower than the fixed pool, and the other tools reached the time limit in 6 of 20 rows. On the eight large runs the medians were 2,561 s (10.1 MB/s) for the default and 699 s (37.0 MB/s) for the fixed pool, against 1,715 s (5.4 MB/s) for Kingfisher, the fastest other tool by median; the paired ratio of the default to the fastest complete tool was 5.8, but not in every repetition: in two of five, iSeq `-p 8` finished first. The gain comes from connections: adaptiSeq held a median of 15.6 (small runs) and 21.7 (large runs) open connections to ENA, against one for iSeq and fastq-dl and medians of 1.7 and 4.0 for Kingfisher. Fixing the number of files in flight from 1 to 64 shows near-proportional gains up to eight and 15–23 times over one file at a time at the best setting (Supplementary Fig. S1).

Completeness. adaptiSeq delivered all runs with every file verified in 16 of its 20 rows (Fig. 1b). In the other four, one run of the 241 was missing; ENA's copy of one file failed MD5 three times or could not be opened, adaptiSeq exited 1 and named the run in `fail.log`. Over the five repetitions it verified 1,203 of 1,205 small-run downloads and 40 of 40 large-run downloads, never delivered a file it could not check, and never exited 0 with a run missing (0 of 20 rows). iSeq, Kingfisher and fastq-dl verified 67–85% of small-run downloads and 58–98% of large-run downloads, exited 0 with runs short in 8 of their 40 rows, and delivered 204 run downloads as files that cannot be checked against ENA's MD5, almost all because ENA's API answered without FASTQ links and the tool fetched the run from SRA instead. ENA also served bad copies: 178 MD5 mismatches appeared in the logs of 44 of 96 benchmark rows, in every tool and on different files in different arms of the same hour, so detecting and re-fetching them in the pool, rather than at the end of a run, is part of the speed as well as the correctness.

Reliability. In separate tests of an earlier build (0.1.3, single-stream transfers, one machine, six trials per tool), adaptiSeq and iSeq both resumed a transfer killed at 25, 50 or 75% without re-downloading completed bytes; in adaptiSeq a dropped connection never produced a truncated output file and a corrupted byte was detected and repaired (Supplementary Table S1).

4 Discussion

adaptiSeq is aimed at lists, where concurrency across runs and within files, and an exit status that can be trusted, matter more than the speed of one transfer; for a single large file, a multi-connection downloader such as aria2c is likely as fast. Throughput from the archives changed more than tenfold between days (the per-connection rate from our node ranged from 0.5 to 7 MB/s), so absolute numbers describe one site over one week and only paired ratios within a repetition are comparable. A fixed pool of 20 was faster than the adaptive default on most ENA cases (median ratio 1.6 and 2.0 on small and large runs), and the default was held back in some repetitions by a single failed transport probe that sent a whole list to FTP. adaptiSeq is distributed through PyPI and not yet Bioconda, and relies on external programs (`wget`; sra-tools and `pigz` when conversion is needed).

Acknowledgements

adaptiSeq is derived from iSeq (Chao *et al.*, 2024) and its transfer engine from FastBioDL (Swargo *et al.*, 2025). This work used Expanse at the San Diego Supercomputer Center through allocation CIS250262 from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program (Boerner *et al.*, 2023), supported by U.S. National Science Foundation grants #2138259, #2138286, #2138307, #2137603 and #2138296.

Funding

[Funding statement to be completed by the authors.]

Conflict of interest

None declared.

Data availability

Accession lists, checksum manifests, benchmark scripts, raw result tables and the analysis scripts are in the project repository. All sequencing data are public in ENA and SRA.

References

Ara,T. *et al.* (2026) DDBJ update in 2025: system integration for global data-sharing including pathogen surveillance. *Nucleic Acids Res.*, **54**, D106–D110.

Boerner,T.J. *et al.* (2023) ACCESS: advancing innovation. In *Practice and Experience in Advanced Research Computing (PEARC '23)*, pp. 173–176.

Chao,H. *et al.* (2024) iSeq: an integrated tool to fetch public sequencing data. *Bioinformatics*, **40**, btae641.

Chen,T. *et al.* (2021) The Genome Sequence Archive Family: toward explosive data growth and diverse data types. *Genomics Proteomics Bioinformatics*, **19**, 578–583.

Choudhary,S. (2019) pysradb: a Python package to query next-generation sequencing metadata and data from NCBI Sequence Read Archive. *F1000Research*, **8**, 532.

Ewels,P.A. *et al.* (2020) The nf-core framework for community-curated bioinformatics pipelines. *Nat. Biotechnol.*, **38**, 276–278.

Katz,K. *et al.* (2022) The Sequence Read Archive: a decade more of explosive growth. *Nucleic Acids Res.*, **50**, D387–D390.

Patel,H. *et al.* (2026) nf-core/fetchngs v1.13.0. Zenodo, doi:10.5281/zenodo.22764959.

Petit,R.A. III *et al.* (2026) fastq-dl: efficiently download FASTQ files from SRA or ENA repositories, v4.0.1. Zenodo, doi:10.5281/zenodo.8051230.

Strande,S. *et al.* (2021) Expanse: computing without boundaries. In *Practice and Experience in Advanced Research Computing (PEARC '21)*, pp. 1–4.

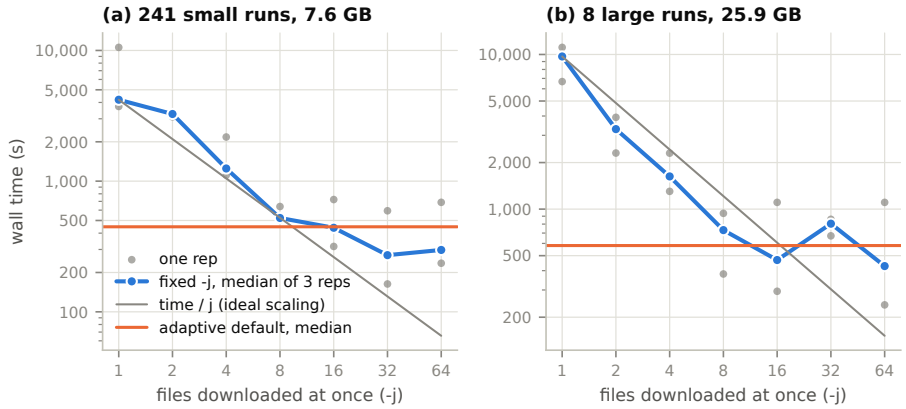
Swargo,R.M., Arslan,E. and Arifuzzaman,M. (2025) Adaptive parallel downloader for large genomic datasets. arXiv:2508.05511.

Woodcroft,B.J. *et al.* (2024) Kingfisher: a utility for procurement of public sequencing data. Zenodo, doi:10.5281/zenodo.10525085.

Yuan,D. *et al.* (2026) The European Nucleotide Archive in 2025. *Nucleic Acids Res.*, **54**, D120–D127.

Supplementary data

Supplementary material to "adaptiSeq: complete, verified download of public sequencing accession lists". Submitted as a separate file with the journal. Full methodology, every benchmark row and the per-campaign figures are in the repository (`plans_and_experiments/paper/methodology`).

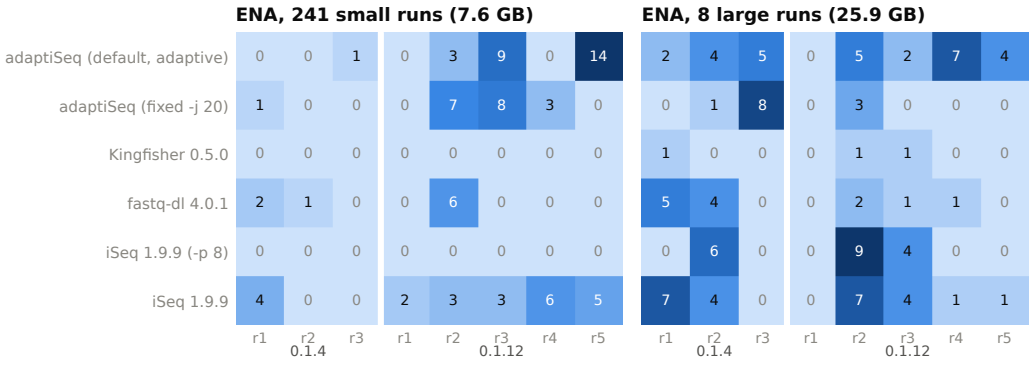


Supplementary Fig. S1. Wall time against the number of files in flight, fixed `-j` from 1 to 64 (build 0.1.7, three repetitions; grey dots are single repetitions, blue is the median, the grey line is perfect scaling from `-j` 1, the orange line is the adaptive default). On the 241-run list, one file at a time took 4,197 s and the best median (`-j` 32) 271 s (15 times faster); on the eight large runs, 9,708 s and 428 s (`-j` 64; 23 times).

Supplementary Table S1. Resume after a kill (404 MB file, single stream; build 0.1.3, one 8-core machine, two repetitions per kill point).

Tool	Killed at	Outcome	Bytes re-downloaded	Final MD5 valid	Trials
adaptiSeq	25, 50, 75%	resumed from offset	0%	6/6	6
iSeq	25, 50, 75%	resumed from offset	0%	6/6	6

Check (adaptiSeq)	Condition	Observed
Never truncate	server without range support	single stream; complete file, MD5 match (2/2)
Short read	connection dropped mid-body	truncated `.part` never renamed (2/2)
Corruption	one byte flipped after download (live ENA)	caught by MD5; re-download valid (1/1)
Circuit breaker	synthetic HTTP 429	host limit halved per trip, backoff 1, 2, 4, 8 s; MD5 valid (2/2)



Supplementary Fig. S2. MD5 mismatches logged per arm and repetition in the first full run (build 0.1.4, left of each panel) and the final comparison (build 0.1.12, right). Mismatches move between tools and days: they are bad copies or bad transfers at the archive. Counts come from log lines whose wording differs between tools.